

Topic 3: BamFeatures

Lab Course Genome-Oriented Bioinformatics

December 2024

This report presents the BAMFeatures program, a tool developed for the classification of RNA sequencing (RNAseq) read pairs based on their alignment to the genome by a mapper. BAMFeatures categorizes read pairs into classes such as **genic**, **intergenic**, and **intergenic containing**, with further distinctions for genic reads as **transcriptomic**, **merged-transcriptomic**, or **intronic**. These classifications are determined by evaluating whether read pairs are fully contained within genes, align contiguously to specific transcripts, match to merged transcripts, or reside within intronic regions. The formal definitions of each category are detailed and the biological plausibility of read pair mappings is discussed. Additionally, the runtime performance of BAMFeatures across different datasets is benchmarked, demonstrating its scalability and efficiency in processing large BAM files. Finally, The results of the program are analyzed and the distribution of Reads Per Kilobase of transcript per Million mapped reads (RPKM) values is presented.

Contents

1	Introduction	2
1.1	SAM Format	2
1.2	Read Mapping Types	2
1.3	PCR Amplification	3
1.4	Test Files	4
1.5	Gene Expression Units	4
2	Program Structure	4
2.1	Program Usage	4
2.2	Program Procedure	5
2.3	Output Format	7
3	Analysis	8
3.1	Correctness	8
3.1.1	RPKM Distribution	8
3.1.2	Annotation Counts	9
3.1.3	PCR Index Distribution	10
3.1.4	Gene Distance Distribution	10
3.1.5	JUnit tests	11
3.1.6	Feature Counts	11
3.2	Time	12
4	Supplement	12
4.1	Feature Counts Usage	12

1 Introduction

RNA sequencing (RNAseq) is a powerful and widely used method for analyzing the transcriptome, providing insights into gene expression patterns, alternative splicing, and the discovery of novel transcripts. By converting RNA into complementary DNA (cDNA) and sequencing it using high-throughput platforms, researchers can quantify and characterize the complete set of RNA transcripts (those that are sequencable) present in a sample. The results of an RNAseq experiment are a set of sequenced reads in the FASTQ format, either single-end or paired-end. These reads are then mapped back onto the reference genome using a mapper such as **STAR** [1].

A typical RNAseq workflow includes sample extraction, sequencing, mapping, annotation, and downstream analyses such as differential expression analysis. In a perfect scenario, each read originates from exactly one transcript. However, sometimes it is unclear from which transcript or even gene a read originates. Additionally, annotations can be incomplete, sequencing or mapping errors can occur, and genes or transcripts can overlap, complicating the mapping process. The annotation steps attempt to quantify and correct these errors, sort out improbable or incorrect reads, and assign reads to genes.

Strand specificity in RNA sequencing refers to the ability to determine the strand of DNA from which an RNA transcript originates. This property is important because transcription is inherently directional, with RNA synthesized from only one DNA template strand. Strand-specific sequencing protocols preserve this directionality, allowing researchers to distinguish between overlapping genes transcribed on opposite strands or sense and antisense transcription. The need for strand specificity arises from the complexity of eukaryotic genomes, where overlapping genes, antisense transcription, and non-coding RNAs are common. It can be challenging to accurately assign reads to their source transcripts without strand-specific information, leading to ambiguous or erroneous interpretations. By maintaining strand specificity, RNAseq enables a more precise understanding of transcriptional regulation, splicing, and gene expression patterns. Thus, some of the experiments used in this project will be strand-specific while others are not.

1.1 Sequence Alignment Map Format

The Sequence Alignment Map (SAM) format is a widely used text-based format for storing biological sequences that are aligned to a reference genome. Each entry in a SAM file contains detailed information about a read and its alignment, such as the alignment positions and possible mismatches. While SAM files are human-readable, they are relatively large in size, as each field is stored as plain text.

BAM files are the binary equivalent of SAM files, designed to store the same alignment information in a compressed and efficient manner. Unlike the plain-text SAM format, BAM files use binary encoding to represent data, significantly reducing their storage requirements. For example, when storing numbers in plaintext encoded in UTF-8, the number '64' would be encoded as the characters '6' and '4', each requiring 8 bits. In contrast, the binary representation of 64 would be 01000000, which is 8 bits in total. This is an oversimplified example but it illustrates some of the advantages of the binary format. BAM files can be indexed and sorted by name or position, enabling fast access to specific regions of interest. However, this means that a single forward/reverse read pair might not appear consecutively, as the reads are sorted by their position on the reference genome. Several read inconsistencies occur in BAM files that should be excluded from analysis:

- Reads in a pair mapping to different chromosomes.
- Both reads in a pair mapping to the same strand.
- Overlapping reads in a pair that imply different introns (see Figure 1).

When annotating, the goal is to identify the most plausible explanation for each read based on its alignment and features. A read may originate from a transcript, reflecting active transcription within an annotated gene, or from a gene's transcribed region but not its exons, such as an intronic region or a novel isoform—especially if splicing events are observed. Alternatively, reads may align antisense to known transcripts, potentially indicating by-products or genuine antisense transcription. Some reads may be classified as intergenic, often aligning near gene boundaries. This could signal novel isoforms or unannotated untranslated regions (UTRs).

1.2 Read Mapping Types

To accurately classify read pairs, it is essential to define the different types of mappings. A consistent read pair can be classified as **genic**, **intergenic**, or **intergenic containing**. A genic read pair is one where both reads in a pair are contained within a gene. Formally, this means all mapped bases of the read are greater than or equal to the gene start position and less than or equal to the gene end position (inclusive). A read pair is **intergenic** if it is not **genic** and does not completely contain one or more genes. A read contains a gene if the first mapped base of the read is less than or equal to the start of the gene and the last mapped base is greater than or equal to the end of the gene. All other read pairs are **intergenic-contained**.

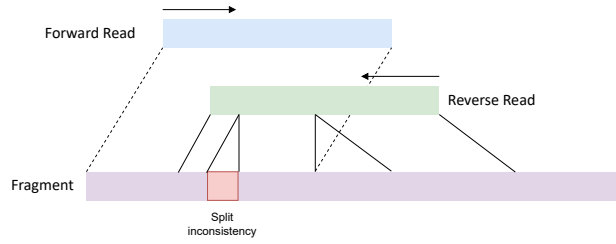


Figure 1: Diagram showing an example of an inconsistent read where the reads in a pair overlap but imply different introns (marked in red).

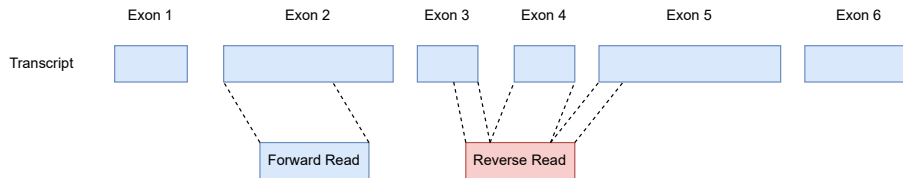


Figure 2: Diagram showing an example of a transcriptomic read pair. The read pair is fully contained within a gene and matches contiguously to a specific transcript section. The color indicates the strand.

A **genic** read pair can be further classified as **transcriptomic**, **merged-transcriptomic**, or **intronic**. A read pair is considered **transcriptomic** if it is fully contained within at least one annotated transcript and aligns contiguously to a specific transcript sequence section (see Figure 2). This means that the intron boundaries observed in the read pair are consistent with those defined in the transcript annotation. Formally, a read pair matches a transcript if both read Region Vectors (RVs) are sub-RVs (defined later) of at least one transcript.

A read pair is classified as **merged-transcriptomic** if it aligns to the merged transcript of a gene but does not necessarily match contiguously (Figure 3). The merged transcript represents the union of all exonic regions of a gene, disregarding specific intron boundaries. This classification accounts for reads that may span novel isoforms or unannotated transcripts. Formally, a read pair matches the merged transcript if each base of the read pair aligns with some exon in the merged transcript. This broader definition enables the identification of potential new transcript variants while maintaining consistency with the gene's exon structure.

A read pair is classified as **intronic** if it is fully contained within the gene body but does not align to any annotated transcript or merged transcript (Figure 4). Intronic reads may represent unprocessed RNA, novel isoforms with unannotated intron retention, or other transcriptional events within the gene body.

Since we are only looking for the most biologically plausible explanation, we only want to consider the most plausible case (**transcriptomic** > **merged-transcriptomic** > **intronic**). For example, if a read pair matches a specific transcript of one gene but also the merged transcript of another gene, we would only consider the transcriptomic match as it is more plausible. If a read pair matches multiple transcripts of multiple genes, we would want to know all the matched transcripts.

1.3 PCR Amplification

Polymerase Chain Reaction (PCR) amplification is a crucial step in RNA sequencing workflows to increase the quantity of DNA fragments before sequencing. However, PCR amplification can introduce biases, such as the preferential amplification of certain fragments, leading to overrepresentation of some reads in the dataset. A possible reason for this is because the process is exponential, if a fragment is amplified, it increases the chance that it will be amplified again. This continues exponentially and can lead to the overrepresentation of certain fragments. These PCR duplicates can affect downstream analyses, including quantification of gene expression

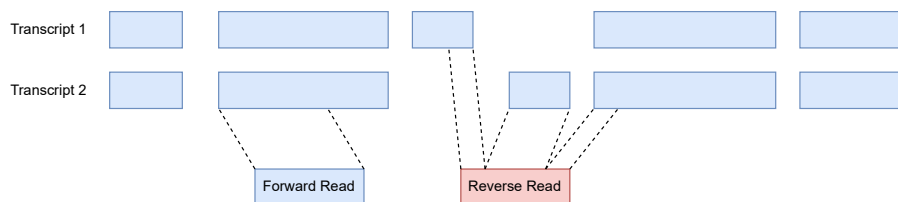


Figure 3: Diagram showing an example of a merged transcriptomic read pair. The read pair aligns to the merged transcript of a gene but does not necessarily match contiguously. The color indicates the strand.

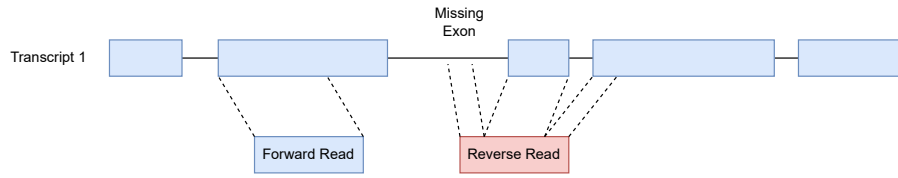


Figure 4: Diagram showing an example of an intronic read pair. The read pair is fully contained within the gene body but does not align with any annotated transcript or merged transcript. The color indicates the strand.

levels. The PCR index (`pcrindex`) is a metric used to identify read pairs that may have been amplified multiple times due to PCR cycles. It is calculated by tracking all observed read pairs and their associated regions (e.g., Region Vectors or RVs). If a read pair's Region Vector matches previously observed regions, it is flagged as a potential PCR duplicate. This approach ensures that repeated fragments, whether arising from overamplification or technical artifacts, are systematically identified. There are multiple methods that are frequently used to address this distortion. Downsampling simply involves downweighting overamplified fragments. Barcoding fragments before PCR allows us to distinguish true biological duplicates from ones created due to PCR amplification. In this project, we will compare the results of all reads and those that are unique.

1.4 Test Files

To optimize computation time, the submission server will verify correctness using small test files. However, for evaluation, three large complete BAM files are provided. The objective of the project is to process the `ebna_hisat.bam` file within one hour while using a maximum of 10GB of RAM. Additionally, the annotations of the read pairs in the test files will be analyzed. The other files used in this assignment are shown in Tables 3 and 4.

1.5 Gene Expression Units

Gene expression levels in RNA sequencing are often quantified using standardized units that account for sequencing depth and gene length. One commonly used metric is Reads Per Kilobase of transcript per Million mapped reads (RPKM), which normalizes read counts to enable comparisons across genes and samples (Equation 1)[2]. This metric is calculated per gene.

$$RPKM = \frac{r \times 10^3 \times 10^6}{t \times l} \quad (1)$$

Figure 5: RPKM Formula with r being the number of reads mapped to the gene, t being the total number of mapped reads in the experiment, and l being the length of the gene in base pairs (merged transcriptomic length). The factors 10^3 and 10^6 are used for scaling: 10^3 to convert the length into kilobases (kb) and 10^6 to normalize the read count per million mapped reads.

2 Program Structure

2.1 Program Usage

```
usage: java -jar bamfeatures.jar [-h] -gtf <gtf_file> -bam <bamfile>
      -o <output_tsv> [-frstrand <true/false>]
```

named arguments:

<code>-h, --help</code>	show this help message and exit
<code>-gtf <gtf_file></code>	GTF file
<code>-bam <bamfile></code>	BAM file
<code>-o <output_tsv></code>	Output file
<code>-frstrand <true/false></code>	true/false

- i. If yes, add the records in the correct order (first of pair) to the readsToAnnotate List
 - ii. If no, add the record to the lookup table and continue to the next record.
- (e) Go to step 2a.

3. New Chromosome Processing :

(a) Read Processing:

- i. Start two threads: Producer and Consumer.
 - ii. Producer iterates through the reads:
 - iii. Check the **cgenes** and if empty then **igenes** for the read pair.
 - iv. If the read pair is genic (**cgenes** > 0), check if it is **transcriptomic**, **merged-transcriptomic**, or **intronic**.
 - v. If the read pair is intergenic (**cgenes** = 0 $\wedge$ **igenes** = 0), get the distance to the next gene and check if the read pair is antisense matching
 - vi. Send the read pair to the consumer thread and go to step 3(a)ii.
- (b) Reset the data structures and return to step 2b.

4. Last Chromosome Processing :

- (a) Process the reads like in Step 3a. Close the writer.

During initialization, all the required data structures for processing are set up. Depending on whether the experiment is stranded or unstranded, the interval tree searching and PCR index processing are slightly different. However, these differences are only in the implementation, not in the core functionality. This provides an excellent opportunity to use an interface. Two interfaces are used: **IntervalTreeForestManager** and **PCRIndexManager**. Each has two implementations—one for strand-specific experiments and one for unstranded experiments.

In a strand-specific experiment, two trees are used for each chromosome: one for each strand. This ensures that only the relevant strand is searched for a given read. In contrast, for an unstranded experiment, only one tree is needed, as both strands are always searched for the best match. As a result, the methods calculating **igenes**, **cgenes**, and **gdist** are all slightly different, but they essentially perform the same search. The **isAntisenseBetter** method always returns **false** in unstranded experiments, since both strands are already checked, whereas in stranded experiments, it performs an actual check. The **PCRIndexManager** also differs between stranded and unstranded experiments. In stranded experiments, it is important to ensure that the read maps to the exact location, including the strand. This requires maintaining two cache maps—one for each strand. The PCR index itself is organized as a **TreeSet** of region vectors, each corresponding to reads mapped to the PCR index. Crucially, the region vectors need to be "melted" (as explained later), because an alignment of (5,10) and (10,15) is considered equivalent to (5,15), and these should be counted together. When iterating through the **SAMRecords**, if the read is from a new chromosome, all reads from the previous chromosome are processed. Since the file is organized by chromosome and position, the PCR index and lookup data structures can be reset. The reads are processed using a producer-consumer pattern. Each record is then checked for validity. A read is considered valid only if all the following conditions are met:

1. The read is mapped
2. The read is part of a pair
3. The mate read is mapped
4. The reads are on different strands
5. The reads are on the same chromosome
6. The read is not a supplementary alignment

This check ensures that only valid reads are processed, guaranteeing the correctness of the annotation. Since both records of the first and second read in a pair are needed, processing cannot begin until both reads are found. A lookup table is used to match reads with their mates. If the mate has not been found yet, the read is added to the table using the read name, which is the same for paired reads but unique across the experiment. Once the mate is found, both reads are added to the **readsToAnnotate** list in the correct order. After processing the entire chromosome, all pairs in **readsToAnnotate** are annotated, and any remaining entries in the lookup table are ignored.

The annotation of a read pair begins with extracting the alignment's start and end positions. These coordinates are then used to find the genes on the current chromosome and strand that fully include the read pair in

their intervals. If any matching genes are found, the read pair is checked for genic properties. First, the alignment blocks of both reads are saved and "melted"—combining overlapping or neighboring blocks into a single block. For two region vectors, x and y , if $x_{\text{end}} \geq y_{\text{start}}$, a new region vector z is created as $z := (x_{\text{start}}, y_{\text{end}})$, for all combinations of x and y . This ensures edge cases are handled correctly when calculating the PCR index, matching genes, and identifying splits. Afterward, consistency is checked by verifying that neither read contains a base from the implied intron of the other (see Figure X). This step also calculates the number of splits, which is saved directly. If the pair is consistent, basic read information such as clipping, mismatches, and PCR index is calculated.

Next, the read is classified as transcriptomic, merged-transcriptomic, or intronic. To classify a read as transcriptomic, both reads' region vectors must be sub-region vectors of at least one transcript. A read region vector r is considered a sub-region vector of a transcript t if $\text{cut}(t, r_{\text{start}}, r_{\text{end}}) = r$, as shown in Figure 7. All matching transcripts are outputted, and the result of the cut operation is compared with the read vectors. If no match is found, a search for a merged transcript match is performed. A merged transcript is the union of all exonic regions of a gene. If both reads are contained within the merged transcript, the read is classified as a merged transcriptomic match. "Contained" means that every base of the read aligns to some exon in the merged transcript (see Figure 8). If no merged transcript match is found, the read is classified as intronic.

If no gene or merged transcript contains the read, an intergenic check is performed. The interval tree is searched to see if the read pair fully contains a gene. If so, the read is ignored. The same checks as for the genic case are applied, but the gene distance is also calculated, and antisense matches are checked. Gene distance is determined by finding the nearest gene on the same strand, while antisense matches are checked by searching the interval tree for the antisense strand. If a genic match is found on the antisense strand, the read pair is classified as an antisense match. The PCR index is calculated by looking up the region vectors in the cache map. If the region vectors are found, the PCR index is outputted and iterated. Finally, the read pair is outputted.

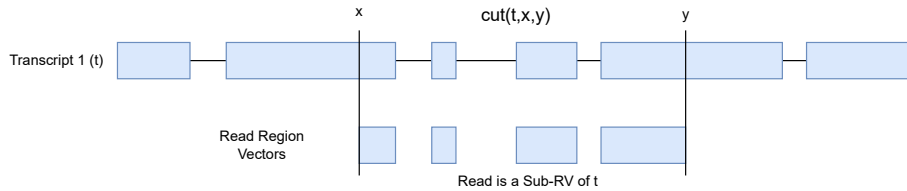


Figure 7: Diagram showing how the cut method works for checking if a read matches a transcript. The transcript is cut starting from the first base of the read and ending at the last base of the read. If the cut transcript is the same as the read, the read matches the transcript.

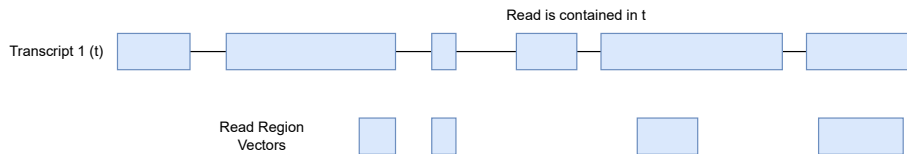


Figure 8: Diagram showing how contained works for checking if a read matches a merged transcript. The read is contained in the merged transcript if each base of the read aligns to some exon in the merged transcript.

2.3 Output Format

The output is structured in a TSV, with each line representing one read pair with the first column representing the read ID. The rest of the annotation is dependent on what type of annotation it is. Only valid reads that are **genic** or **intergenic** are outputted. If they are inconsistent we annotate with `split-inconsistent:true`. If not, we annotate the general statistics:

- **Mismatch count:** `mm:<mismatch-count>`
- **Clipping size:** `clipping:<clipping-count>`
- **Split count:** `nsplit:<split-count>`
- **Gene count:** `gcount:<gene-count>`

If the read is genic, we also output the following depending on the match level:

- **Transcriptomic:** `<gene-id1>,<gene-biotype1>:<transcript-id1>,...[<gene-2...>]`

- **Merged-Transcriptomic:** `<gene-id1>,<gene-biotype1>:MERGED|[gene-2...]`
- **Intronic:** `<gene-id1>,<gene-biotype1>:INTRON|[gene-2...]`

If it is intergenic, we output the gene distance (the distance to the next sense gene): `gdist:<gene-distance>` and whether the read pair matches a gene on the other strand `antisense:true|false` (If it is a strand unspecific experiment, it is always false). Additionally, we output the PCR index of the read pair (i.e. the number of read-pairs mapped exactly to the same genomic region as this read pair so far): `pcrindex:<pcrindex>`.

3 Analysis

The following analysis was performed with the sample files `ebna_hisat.bam` and `hes_star.bam` using `Homo_sapiens.GRCh37.75` and `nookaew_cm.bam` using `Saccharomyces_cerevisiae.R64-1-1.75.gtf`. The program was run using the specified strandness from the reference table on the Biocluster grid. The files were queued with `QueueGridForRefTable.sh` and analyzed with `QueueGridForAnalyze.sh` using `bamfeatures.jar` and `AnalyzeBAMFeatures.py` respectively.

3.1 Correctness

3.1.1 RPKM Distribution

The RPKM values were calculated for all genes mapped in the three provided analysis files using the formula in Figure 1. The RPKM values were calculated for all reads and for unique reads (non-PCR amplified reads) separately. The distribution of the RPKM values was then plotted on a log scale for each file. The distribution of the RPKM values for all reads, and unique reads are shown in Figure 9. The distribution of the RPKM values for the top 10 genes in each file is shown in Figure 10. The comparison of the RPKM values for all reads and unique reads is shown in Figure 11. We can see that the unique RPKM values are lower than the total RPKM values with `hes_star` and `ebna_hist` having a larger difference than `nookaew_cm`.

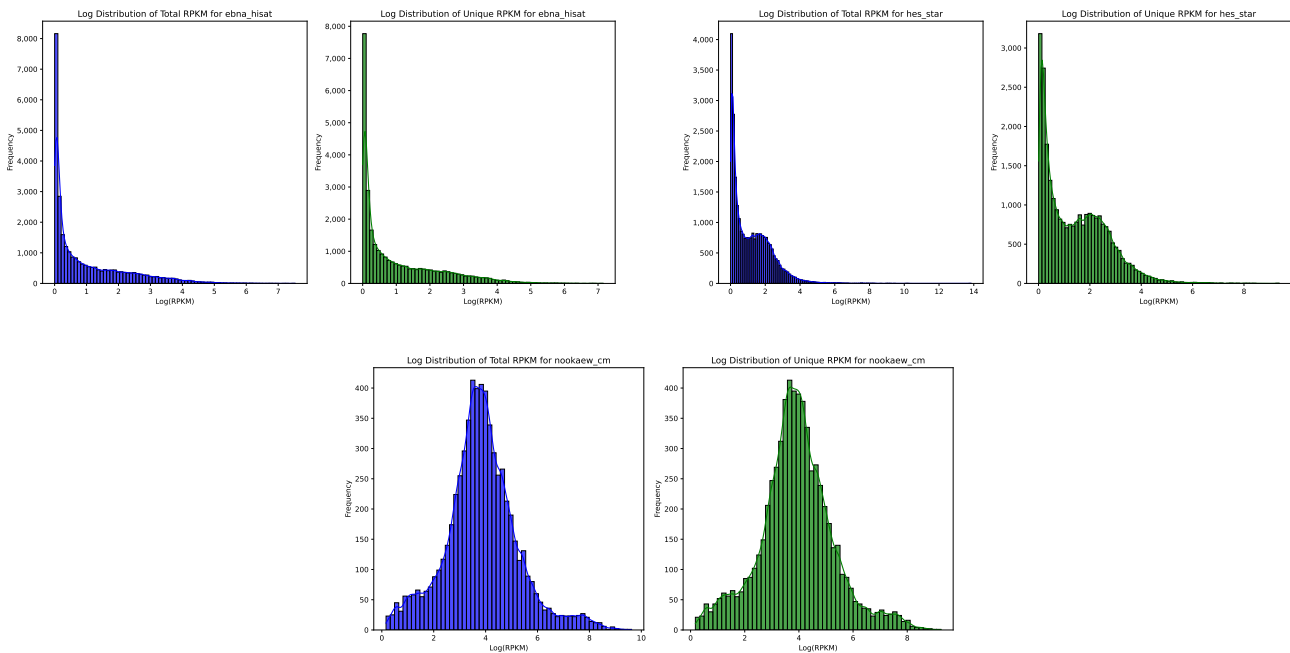


Figure 9: Log distribution of the total (all reads) vs unique (non-PCR amplified) RPKM values for the provided analysis files with the kernel density function overlayed. Bin size is determined by the minimum of the sturges and fd estimators. The RPKM values get smaller in the unique case.

Figure 9 presents the log-transformed distribution of RPKM values for all genes in each dataset, overlayed with a kernel density estimate (KDE). The bin size was chosen based on the minimum of the Sturges and Freedman-Diaconis estimators to achieve an optimal balance between resolution and interpretability. The unique (non-PCR amplified) RPKM values are lower than the total RPKM values, indicating the removal of duplicated reads likely due to PCR amplification. The magnitude of this difference varies across datasets, reflecting differences in sequencing quality, depth, and library preparation protocols. However, it demonstrates the importance of this correction step in ensuring accurate gene expression quantification while noting that the unique RPKM values are overcorrecting for true biological replicates (as explained earlier). For the following analysis, we will only consider the result genes

for the unique RPKM values (Figure 10, green plot). The top gene for the **hes_star** experiment, [ENSG00000210082](#), codes for the mitochondrial 16sRNA. The top few genes for **nookaew_cm** are all related to Glycolysis like [YGR192C](#) which codes for Glyceraldehyde-3-phosphate dehydrogenase (GAPDH), [YKL060C](#) which codes for Fructose 1,6-bisphosphate aldolase, and [YHR174W](#) which codes for ENolase. Similarly to **hes_star**, the top genes for **ebna_hist** are mostly noncoding RNAs, such as [ENSG00000249335](#) and [ENSG00000256725](#) which are LincRNAs.

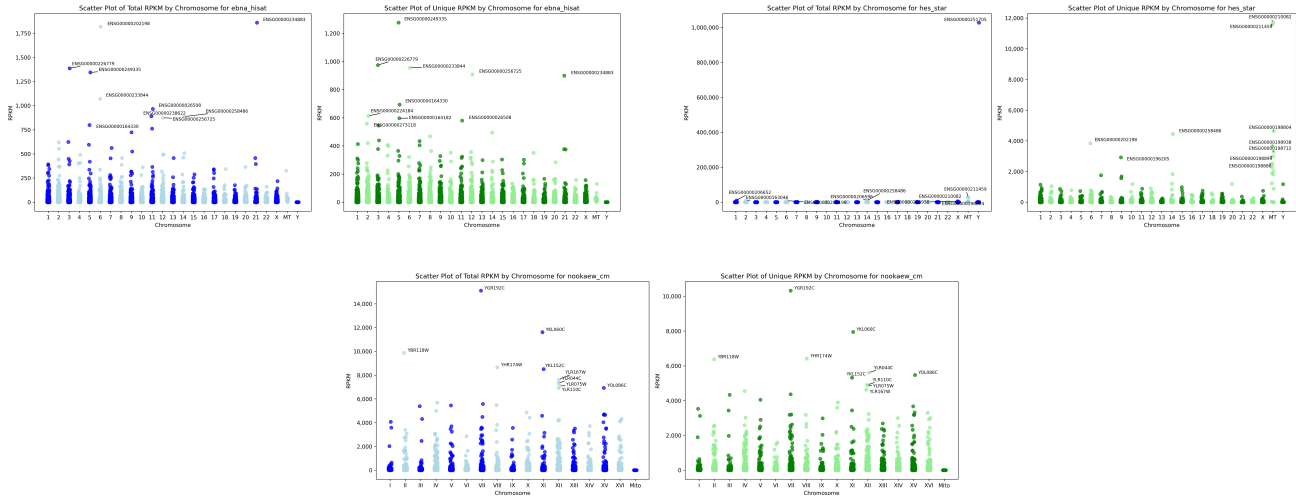


Figure 10: Scatter plot of the total (all reads) vs unique (non-PCR amplified) RPKM values for the provided analysis files separated by chromosome. The top 10 genes are labeled. The unique transformation greatly changes **hes_star**, as it has an extreme outlier RN5-8S6, a ribosomal pseudogene. **nookaew_cm** is more consistent and barely changes the top 10, with most being gene coding for proteins in Glycolysis. **ebna_histat** has a few outliers, but the top 10 are mostly the same and also rRNAs.

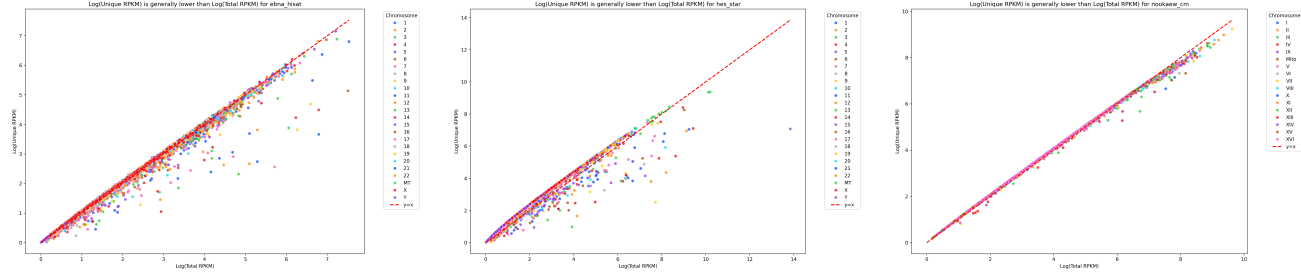


Figure 11: Comparison of log-transformed Unique RPKM and Total RPKM values for the provided analysis files. Each scatter plot represents genes grouped by their chromosomes, with the dashed red line indicating the $y = x$ diagonal, where Unique RPKM equals Total RPKM. Points below the diagonal indicate that Unique RPKM is lower than Total RPKM, reflecting the presence of PCR duplicated reads. The **nookaew_cm** dataset (yeast) shows a smaller reduction in RPKM values compared to **ebna_histat** and **hes_star** (both human), suggesting fewer multimapped reads.

3.1.2 Annotation Counts

The annotation count analysis reveals distinct patterns across the datasets. The **ebna_histat** dataset exhibits a high proportion of intronic reads, suggesting substantial transcriptional activity in non-coding regions, which might indicate alternative splicing events or incomplete processing. The **hes_star** dataset also shows significant intronic read representation but to a lesser extent, implying a less pronounced but still notable non-coding transcriptional activity. In contrast, the **nookaew_cm** dataset, derived from yeast, demonstrates minimal intronic reads, consistent with the compact genome organization of yeast, which features fewer and shorter intronic regions compared to higher eukaryotes. The relatively large amount of intergenic reads could be due to an incorrect annotation or ncRNAs transcribed from intergenic regions.

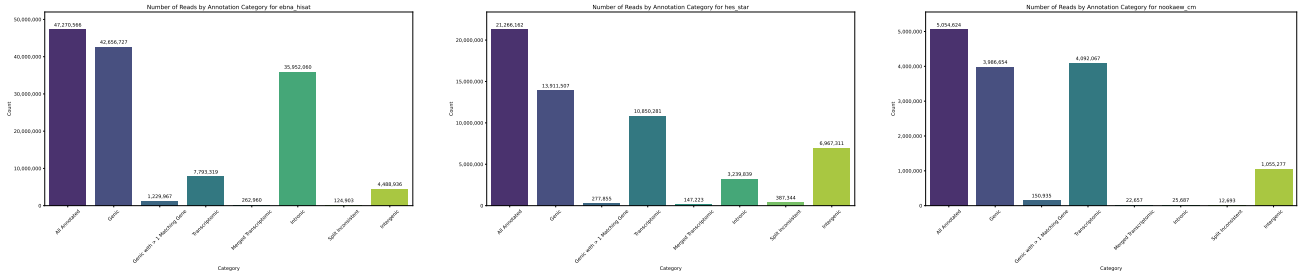


Figure 12: Comparison of read counts across annotation categories for the provided analysis files. Categories include genic regions, intergenic regions, transcripts, introns, and inconsistent annotations, with the total annotated reads being the sum of genic, intergenic, and split inconsistent categories. The *ebna_hisat* dataset exhibits a high proportion of intronic reads, indicating significant transcriptional activity in non-coding regions. Similarly, *hes_star* shows a substantial intronic component, though to a lesser extent. In contrast, the *nookaew_cm* dataset, derived from yeast, contains very few intronic reads, reflecting the compact genome organization of yeast with minimal intronic sequences.

3.1.3 PCR Index Distribution

The cumulative distributions of PCR index counts for three datasets were analyzed, as shown in Figure 13. The *ebna_hisat* experiment spans PCR index counts up to approximately 3,000,000, while *hes_star* reaches over 1,000,000, driven largely by a single outlier read with a PCR index of 1,063,959. In contrast, the *nookaew_cm* dataset has a much smaller range, with PCR index values capped below 200. These differences highlight substantial variability in amplification dynamics across datasets.

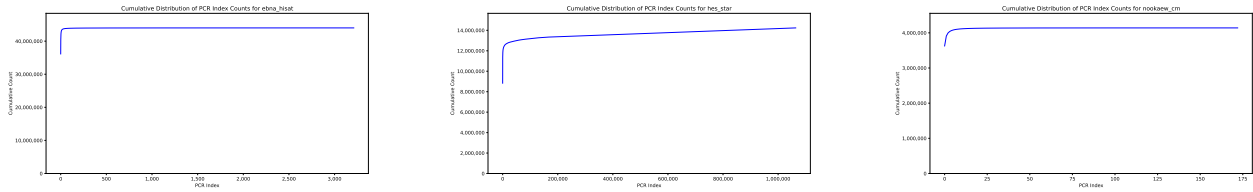


Figure 13: Cumulative distributions of PCR index counts for the provided analysis files. The x-axis represents the PCR index values, and the y-axis shows the cumulative count of reads. The blue line shows the empirical cumulative distribution. Notably, the x-axis scale differs significantly across datasets: *ebna_hisat* and *hes_star* span thousands or hundreds of thousands of PCR index counts, whereas *nookaew_cm* spans fewer than 200. However, the x-axis is extremely distorted in the *hes_star* experiment by single read with a max PCR index of 1,063,959.

To model the PCR Index distributions, various distributions were tested, including the Poisson, negative binomial, and gamma distributions. However, no distribution provided a perfect fit for the data without explicitly modeling the parameters of the distribution after the results annotation. Modeling using the negative binomial distribution fitted better than the other distributions tested, with the parameters r and p calculated through the known variance and mean of the experiment.

3.1.4 Gene Distance Distribution

The cumulative distribution functions (CDFs) of distances from reads to the nearest annotated gene are shown in Figure 14. The x-axis represents the distance to the nearest gene in base pairs, while the y-axis shows the cumulative probability. For all datasets, the majority of intergenic reads are located very close to annotated genes, as indicated by the steep initial rise in the CDFs. This pattern suggests potential annotation issues, such as unannotated intronic retention events or the presence of novel transcripts. The x-axis scale varies significantly across datasets, reflecting differences in genome organization. In the *nookaew_cm* dataset (yeast), distances to the nearest gene are generally much smaller, consistent with the compact nature of the yeast genome. In contrast, the *ebna_hisat* and *hes_star* datasets (human) exhibit larger distances, as expected given the greater prevalence of intergenic regions in the human genome. Despite these differences, the sharp rise in the CDFs near zero for all datasets highlights that most reads align close to genes. This observation supports the notion that many intergenic reads may originate from unannotated genomic features, which could include novel transcripts or retained introns. The large number of annotated reads that have a distance of zero, is largely due to reads

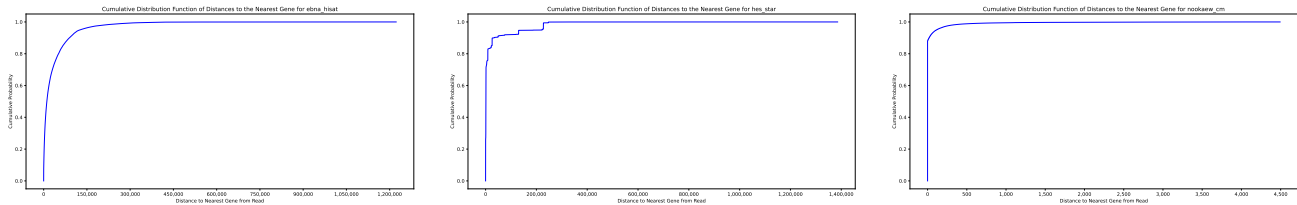


Figure 14: Cumulative distribution functions (CDFs) of distances from reads to the nearest annotated gene for the provided analysis files.. The x-axis represents the distance to the nearest gene in base pairs, while the y-axis shows the cumulative probability. For all datasets, the majority of intergenic reads are located very close to annotated genes, as indicated by the steep initial rise in the CDFs. This suggests the possibility of annotation errors, such as unannotated intronic retention events or novel transcripts. Notably, the scale of the x-axis varies, with *nookaew_cm* (yeast) showing much smaller distances compared to the human datasets (*ebna_hisat* and *hes_star*), consistent with the compact nature of the yeast genome.

Table 1: Reference Table Format. Columns are separated by tabs. Bam: the name of the BAM file. gtf: The corresponding annotation file in GTF format. strandness: Specifies the strandness of the experiment. If the strandness is not given, this column will be empty. reference_solution: The reference solution associated with the experiment.

bam	gtf	strandness	reference_solution
-----	-----	------------	--------------------

that are partially overlapping with the gene. This is due to the way the program calculates the distance, by finding the closest gene to the read pair. If the read pair is partially overlapping with the gene, the distance is zero. This suggests the reliance of gene rich regions for the mapper. The gradual tailing of the distributions in the human experiments displays a subset of reads that are likely 'truly' from intergenic regions, possibly from ncRNAs or other unannotated regions.

3.1.5 JUnit tests

When optimizing the program for performance, this testing suite helps verify that improvements do not compromise correctness. By running this test after performance-related modifications, we ensure that our changes do not affect the accuracy. A JUnit test was implemented in order to test the correctness of the program by comparing it to the reference solutions. In order to run the tests for all provided BAM files with annotations, a reference table format is used (Table 1). The reference solution is read, storing the lines in a map by read ID. When we process a new line in our program's solution, we compare it to the reference solution and remove it from the map. If the annotation does not match, an error is thrown, and the test fails. The testing suite is robust and ignores trivial differences, such as a different order of the listed genes in the transcriptomic case. If we don't find the read ID of the currently processed line of our solution, an error is thrown. Additionally, once we finish processing our solution, if there are still entries in the reference solution map, it means that we are missing a read ID, and the test also fails.

3.1.6 Feature Counts

In order to properly compare the results of the program, the `featureCounts` method in R was used [3]. The tool calculates the count of each feature (gene) per Read Pair. The parameters used for the program are listed in the Supplement 4.1. The results were then converted to RPKM values using the formula in Figure 1. The top 10 genes for each program were then compared. The results are shown in Table 2. The top 10 genes for each program are similar, with some overlap. However, it is important to note that it is not possible to configure `featureCounts` to perform in the same way as our program. For example, merged transcriptomic and intronic read pairs are not assigned to a gene. For example, *ebna_hisat* has a very large proportion of intronic matches, which naturally won't be found by the `featureCounts` method. This can be used as a sanity check but cannot be used to certify the program's correctness. Naturally, there are many ambiguities when solving this problem where it is not clear what should be done with a certain read or what exactly to count into the RPKM value. Likewise, if this assignment were to be solved by 100 talented programmers, you would likely get 100 different but similar answers.

Table 2: Top 10 Total RPKM values obtained from the program described in this paper are obtained from the feature counts of the featureCounts package in R, converted to RPKM. Gene Names in bold mean that the gene is present in the top 10 of the other program, respectively. The left column is the program described in this paper, and the right column is the featureCounts results. There is a significant overlap between the top 10 values.

ebna_hist		hes_star		nookaew_cm	
BAMFeatures	featureCounts	BAMFeatures	featureCounts	BAMFeatures	featureCounts
ENSG00000234883	ENSG00000202198	ENSG00000251705	ENSG00000251705	YGR192C	YLR167W
ENSG00000202198	ENSG00000238622	ENSG00000210082	ENSG00000239776	YKL060C	YKL060C
ENSG00000226779	ENSG00000269900	ENSG00000211459	ENSG00000241781	YBR118W	YDR524C-B
ENSG00000249335	ENSG00000251562	ENSG00000206596	ENSG00000200434	YHR174W	YLR110C
ENSG00000233844	ENSG00000265695	ENSG00000206652	ENSG00000210082	YKL152C	YDR524W-C
ENSG0000026508	ENSG00000266019	ENSG00000163046	ENSG00000209082	YLR167W	YKL153W
ENSG00000238622	ENSG00000265150	ENSG00000202198	ENSG00000211459	YLR044C	YLR076C
ENSG00000258486	ENSG00000263900	ENSG00000258486	ENSG00000265150	YLR075W	YDL133C-A
ENSG00000256725	ENSG00000211593	ENSG00000226958	ENSG00000258486	YOL086C	YKL152C
ENSG00000164330	ENSG00000211595	ENSG00000198804	ENSG00000202198	YLR110C	YHL015W

Table 3: List of the BAM files provided with the respective gtf, experiment strandness, and the runtime (excluding GTF Parsing). The BAM file were processed on the bioclient grid (write/read to/from HDD) with -Xmx10g and queued with QueueGridForRefTable.sh

BAM	GTF	Strandness	Runtime (seconds)	Number of Annotated Reads
ebna_hisat	Homo_sapiens.GRCh37.75	true	325.619	47,270,566
hes_star	Homo_sapiens.GRCh37.75	false	129.077	21,266,162
nookaew_cm	Saccharomyces.cerevisiae.R64-1-1.75	null	29.621	5,054,624

3.2 Time

The runtime of the program depends on 1) the number of records in the BAM File and 2) the size of the GTF file. However, for large BAM files, the processing of the GTF File can be disregarded since the GTF File is a constant size and much quicker to parse than the BAM File. The runtime of the program was measured for the provided BAM files and is shown in Table 3. The runtime of the program was also measured for the BAM files with reference solutions and is shown in Table 4. Compared to **hes_star**, processing **ebna_hist** takes 2.5 times longer, while only having 2.2 more annotations. However, multiple reasons can lead to this discrepancy. Firstly, **ebna_hist** has a far larger proportion of intronic reads, which require far more processing than transcriptomic reads as the merged transcriptome needs to be created. Additionally, **hes_star** has far more split inconsistent reads, even though it is a smaller file. Split inconsistent reads are comparatively quick to process, as they are only checked for consistency and then outputted. Due to this, it is reasonable to conclude that the runtime of the program is linear in the amount of records. The runtime of **nookaew_cm** is much shorter than the human experiments.

4 Supplement

The following sections provide additional details and usage examples.

4.1 Feature Counts Usage

```
counts <- featureCounts(
  files = bam_file,
```

Table 4: List of the BAM files provided that have reference solutions with the respective gtf, experiment strandness, and the runtime (excluding GTF Parsing). The BAM files were processed on the bioclient grid (write/read to/from HDD) with -Xmx10g and queued with QueueGridForRefTable.sh

BAM	GTF	Strandness	Runtime (seconds)	Number of Annotated Reads
y.ns.2	Saccharomyces.cerevisiae.R64-1-1.75	null	0.350	20,876
y.ns.5	Saccharomyces.cerevisiae.R64-1-1.75	null	0.372	18,505
h.sn.1	Homo_sapiens.GRCh37.75	false	0.336	17,209
h.sn.7	Homo_sapiens.GRCh37.75	false	0.340	16,116
h.sp.3	Homo_sapiens.GRCh37.75	true	0.375	22,806
h.sp.8	Homo_sapiens.GRCh37.75	true	0.425	21,092

```
annot.ext = annotation_file,  
isGTFAnnotationFile = TRUE,  
GTF.featureType = "exon",  
GTF.attrType = "gene_id",  
useMetaFeatures = TRUE,  
isPairedEnd = TRUE,  
countMultiMappingReads = FALSE,  
nthreads = 8,  
allowMultiOverlap = TRUE,  
primaryOnly = TRUE,  
strandSpecific = -, % Depends on the experiment 0=null, 1=true, 2=false  
requireBothEndsMapped = TRUE,  
countReadPairs = TRUE,  
countChimericFragments = FALSE,  
)
```

References

- [1] Alexander Dobin et al. “STAR: ultrafast universal RNA-seq aligner”. In: *Bioinformatics* 29.1 (Oct. 2012), pp. 15–21. DOI: <https://doi.org/10.1093/bioinformatics/bts635>.
- [2] Renesh Bedre. *Gene expression units explained: RPM, RPKM, FPKM, TPM, DESeq, TMM, SCnorm, GeTMM, and ComBat-Seq*. Feb. 2021. URL: https://www.reneshbedre.com/blog/expression_units.html.
- [3] Y. Liao, G. K. Smyth, and W. Shi. “featureCounts: an efficient general purpose program for assigning sequence reads to genomic features”. In: *Bioinformatics* 30.7 (Nov. 2013), pp. 923–930. DOI: <https://doi.org/10.1093/bioinformatics/btt656>.